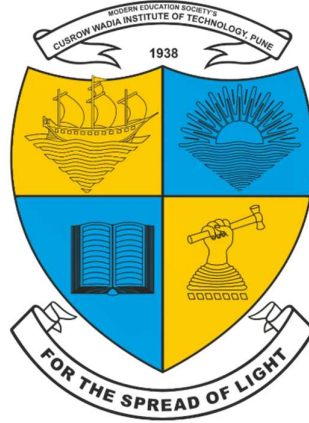


A

Project Report On BLE-Based Employee Tracking System



Submitted to:

Department of Computer Engineering And IOT,
Cusrow Wadia Institute of Technology, Pune – 411001

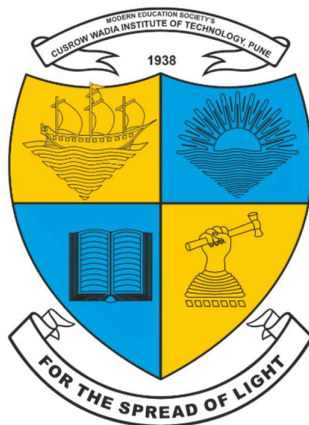
Under the guidance of :

Ms. A.P Rajebhosale

Submitted by:

1. Anmol Umesh Patil – 225040
2. Manthan Ramraje Patil -225044

CERTIFICATE



This is to certify that the below mentioned TY Diploma Students Have Carried Out the project work on **“BLE-Based Employee Tracking System”** in the department of Computer Engineering And IOT, Cusrow Wadia Institute of technology Pune 411001. They Have Completed these project work under My Guidance in Satisfactory manner in may-2025 of Third Year Diploma.

1. Anmol Umesh Patil – 225040
2. Manthan Ramraje Patil -225044

Computer Engineering and IOT have successfully completed project on BLE-Based Employee Tracking System Towards the fulfilment of their Diploma In Computer Engineering and IOT In Academic Year 2024-25.

The performance of each student during the course was very good.

Date:05-06-2025

Ms. A.P Rajebhosale
(Guide)

Dr. S.R Khonde
(H.O.D Comp & IOT)

Dr. A.S Chandak
(Principal)

TABLE OF CONTENTS

Sr. No.	Contents	Page No.
1	Introduction	5
2	Acknowledgement	6
3	Abstract	7
4	Keywords	8
4.1	Definitions, Acronyms, and Abbreviations	8
5	Need of such System	9
6	Detailed Problem Statement	10
7	Software and Hardware Requirements	11
7.1	Software Requirements	11
7.2	Hardware Requirements	12
8	Literature Survey	13
9	Scope of the project	14
10	System Design	15
10.1	ER Diagram	15
10.2	Use Case Diagram	16
10.3	Class Diagram	17
10.4	Data Flow Diagram	18
11	Salient Features	19
12	Code Implementation	21
13	Hardware Preview	36
14	Website Preview	37
15	Future Prospect	41
16	Conclusion	42
17	References	43

LIST OF FIGURES

Figure No.	Title	Page No.
10.1	ER Diagram	15
10.2	Use Case Diagram	16
10.3	Class Diagram	17
10.4	Data Flow Diagram	18
13.1	Hardware Preview	36
13.2	Working of Hardware	36
14.1	Login Page	37
14.2	Dashboard - Overview	37
14.3	Dashboard - Detailed View	38
14.4	EmployeeDetails Page	40

In any organization—be it industrial, commercial, or corporate—accurate employee attendance and work hour tracking are essential components of effective workforce management. Most companies still rely on traditional methods such as manual registers, biometric systems, or RFID-based attendance systems. These methods, while common, come with several limitations such as human error, buddy punching, limited accuracy, maintenance issues, and a lack of real-time monitoring capabilities.

With the increasing demand for automation and the rapid evolution of Internet of Things (IoT) technologies, it has become necessary to rethink and upgrade the conventional methods of attendance management. Our project titled “**BLE-Based Employee Tracking System**” aims to bridge this technological gap by offering a smart, cost-effective, and scalable solution using Bluetooth Low Energy (BLE) beacons and ESP32 microcontrollers.

The system is designed in such a way that each employee carries a small BLE tag, specifically using the SparkFun NanoBeacon IN100. These tags constantly emit a unique signal (UUID), which can be detected by an ESP32-based BLE receiver placed at the main entrance of the workplace or any designated checkpoint. The ESP32 captures the UUID, logs the exact entry or exit time, and forwards this data to a cloud backend system (e.g., Firebase) via Wi-Fi connectivity.

This data is then processed and displayed on a user-friendly web dashboard that can be accessed by administrators or managers in real time. The system calculates the total active work hours per employee on a daily basis, which helps in maintaining accurate attendance records and streamlining payroll calculations.

Unlike traditional methods, this system provides multiple benefits including reduced human error, improved transparency, increased data accuracy, and reduced administrative workload. It also eliminates the need for physical touch (as required in biometric systems), which makes it more hygienic—especially relevant in a post-pandemic world.

Furthermore, the system’s low-power design ensures that the BLE beacons can last for several months to a year on a single coin cell battery, making it both sustainable and easy to maintain. The ESP32-based receiver is programmable, scalable, and cost-efficient, making it suitable for small, medium, or large enterprises.

In conclusion, the **BLE-Based Employee Tracking System** is an innovative step toward automating workforce management using modern IoT solutions. It reflects the growing trend of smart infrastructure and promises to improve overall operational efficiency, reduce costs, and provide managers with reliable data for better decision-making.

2. ACKNOWLEDGEMENT

It gives us immense pleasure to present this project report on “**BLE-Based Employee Tracking System.**” This project marks a significant learning experience in our academic journey and would not have been possible without the support and encouragement of many individuals.

First and foremost, we would like to express our sincere gratitude to our project guide **Ms. Aarti Rajebhosale** for her constant support, expert guidance, and valuable suggestions throughout the development of this project. Her keen interest, timely feedback, and constructive advice have been instrumental in helping us complete this work successfully.

We are extremely thankful to **Dr. A. S. Chandak**, Principal, and to **Dr. S.R Khonde**, **Head of Department, Computer Engineering and IoT**, for providing us with all the necessary resources and facilities required to carry out our project work.

We would also like to extend our heartfelt thanks to all the faculty members of the **Department of Computer Engineering and IoT**, Cusrow Wadia Institute of Technology, Pune, for their encouragement, motivation, and timely assistance throughout the academic year.

A special word of thanks goes to our classmates and friends who supported us directly or indirectly during various stages of the project. Their constant encouragement helped us to stay focused and motivated.

Last but not the least, we would like to thank our parents and family members for their endless support, understanding, and moral backing, which played a crucial role in the successful completion of this project.

This project has not only enhanced our technical knowledge but also improved our teamwork, problem-solving, and project management skills. It has been a truly enriching and rewarding experience.

The growing demand for accuracy, automation, and real-time monitoring in workforce management has led to the need for advanced employee tracking solutions. Traditional attendance systems such as manual registers, RFID swipes, and biometric devices often suffer from inefficiencies, human error, lack of scalability, and insufficient data transparency. These limitations become even more apparent in industrial or large-scale work environments where constant monitoring and accurate data logging are critical.

To address these challenges, we have developed a **BLE-Based Employee Tracking System**, a smart and efficient approach for automating employee attendance and calculating active work hours using **Bluetooth Low Energy (BLE)** technology. The primary goal of this project is to eliminate manual intervention and provide a reliable solution for attendance monitoring and work hour calculation.

The system operates by assigning each employee a BLE beacon—specifically the **SparkFun NanoBeacon IN100**—which emits a unique UUID signal continuously. These signals are detected by an **ESP32-based BLE receiver** placed at the entry and exit points of the workplace. Once detected, the system logs the precise entry and exit timestamps of each employee.

This data is then transmitted via **Wi-Fi** to a **cloud-based backend (Firebase)**, where it is processed to calculate the total number of active hours worked by each employee. The data is then displayed on a responsive **web dashboard**, accessible to managers and administrators for remote monitoring, analysis, and decision-making. The system is designed to be simple, energy-efficient, scalable, and easy to deploy in any office or industrial setup.

By minimizing human error, eliminating the need for physical contact, and allowing real-time data access, this solution not only streamlines attendance tracking but also significantly improves operational efficiency. Additionally, the system is cost-effective and eco-friendly, reducing the reliance on paper-based registers or complex biometric setups.

The **BLE-Based Employee Tracking System** serves as a practical example of how **IoT (Internet of Things)** technologies can be effectively applied to solve real-world administrative problems. It provides a foundation for future improvements, such as integration with payroll systems, employee notifications, or mobile-based alert systems. This project demonstrates the practical utility of combining hardware (ESP32, BLE beacons) with software (Firebase, web dashboard) to deliver an end-to-end automated solution for employee management.

Bluetooth Low Energy (BLE), ESP32, NanoBeacon, Firebase, Employee Tracking, Web Dashboard, UUID, IoT (Internet of Things), Attendance Monitoring, Workforce Management

4.1 DEFINITIONS, ACRONYMS AND ABBREVIATIONS

- **BLE (Bluetooth Low Energy):** A wireless communication technology designed for short-range communication with very low power consumption. Used in this project for beacon-based employee detection.
- **ESP32:** A low-cost, low-power microcontroller with Wi-Fi and BLE capabilities. It is used as the receiver to detect BLE signals from employee beacons.
- **UUID (Universally Unique Identifier):** A unique code transmitted by each BLE beacon to distinguish it from others.
- **Firebase:** A platform developed by Google for creating mobile and web applications. In this project, it is used as the backend database to store and process attendance data.
- **NanoBeacon (IN100):** A small, energy-efficient BLE transmitter used as an employee tag for broadcasting signals.
- **Web Dashboard:** A browser-accessible interface that displays employee work logs and analytics in real-time.
- **IoT (Internet of Things):** A network of physical devices connected via the internet, capable of collecting and exchanging data. This project is an IoT application.
- **Attendance Monitoring:** The process of automatically logging employee entry and exit times and tracking work duration.
- **Workforce Management:** Strategies and tools used to track, manage, and improve employee performance and productivity.

In today's competitive industrial and corporate environment, organizations constantly seek efficient methods to optimize workforce management and productivity tracking. One of the most critical aspects of this is maintaining accurate employee attendance records and monitoring work hours. Traditional methods such as manual logbooks, RFID cards, or biometric systems are still widely used but come with multiple limitations.

Manual systems are prone to human errors, manipulation, and buddy punching. Biometric devices require physical contact and regular maintenance and may not function reliably in outdoor or rugged environments. RFID systems, while touchless, require the user to scan their card actively, which may not always happen consistently. Moreover, none of these methods provide real-time data access or automatic work-hour calculation without additional software integration.

To address these drawbacks, there is a growing need for a fully automated, contactless, and accurate attendance tracking system that can work efficiently in real-time and support remote data access. The **BLE-Based Employee Tracking System** is designed to fulfill this need.

By leveraging BLE (Bluetooth Low Energy) technology, this system allows passive tracking of employee presence without requiring any manual intervention. Employees carry BLE beacons that automatically broadcast signals, which are detected by an ESP32 receiver as they enter or exit the premises. This ensures accuracy, prevents misuse, and eliminates the need for employees to remember to check in or out.

Additionally, with backend support via Firebase and a centralized web dashboard, the system provides instant access to attendance logs and total work hour calculations. This enhances transparency, helps streamline payroll processing, and allows managers to monitor workforce activity even when off-site.

Given the low power consumption of BLE devices and the scalability of the system, it is also a cost-effective solution suitable for a wide range of applications—from small offices to large construction sites and manufacturing units.

Thus, this system fulfills a real and growing need for smarter, more reliable, and hands-free employee attendance management, making it an ideal fit for modern digital workplaces.

6.DETAILED PROBLEM STATEMENT

Employee attendance and work hour monitoring are essential components of workforce management. However, most traditional systems currently in use—such as manual registers, RFID-based cards, or fingerprint biometric systems—are either outdated, error-prone, or inconvenient in practical usage, especially in large-scale or outdoor environments such as construction sites or warehouses.

These systems suffer from multiple limitations:

- **Manual systems** are prone to inaccuracies and often result in false or missed entries.
- **RFID cards** require employees to actively swipe the card, which may be forgotten or misused (e.g., buddy punching).
- **Biometric systems** are maintenance-heavy, require physical contact, and may be unsanitary or non-functional in dusty or humid environments.

Moreover, most of these systems lack remote accessibility, real-time data synchronization, and automatic work-hour calculation. Data often needs to be manually exported or processed before it can be analysed or used for payroll.

The goal of this project is to develop a **BLE-Based Employee Tracking System** that overcomes these challenges by offering:

- **Automatic detection** of employee entry/exit using BLE beacons without manual check-in or touchpoints.
- **Real-time logging** and syncing of attendance data via Wi-Fi to a cloud backend (Firebase).
- **Calculation of total active work hours** for each employee on a daily basis.
- **Remote accessibility** through a responsive web dashboard for monitoring from any location.

This system will help minimize errors, reduce the administrative burden, increase transparency in attendance records, and improve overall productivity. It will be especially useful for small-to-medium businesses, construction sites, workshops, and offices where managing accurate employee records is often difficult due to the lack of reliable, low-cost automation.

7. SOFTWARE AND HARDWARE REQUIREMENTS

To implement the **BLE-Based Employee Tracking System**, both hardware and software components are required. These components collectively enable real-time data transmission, processing, and visualization.

7.1 SOFTWARE REQUIREMENTS

1. **Arduino IDE**

Used for programming the ESP32 microcontroller with the necessary firmware to detect BLE signals and handle Wi-Fi communication.

2. **Firebase (Google Backend-as-a-Service)**

Cloud database to store employee entry and exit timestamps. Offers real-time data synchronization and scalability.



3. **HTML, CSS, JavaScript (React/Java)**

Used to develop the web dashboard interface to view attendance logs, calculate work hours, and manage employee data.

4. **Firebase Admin SDK / REST APIs**

Enables secure interaction between the web interface and the Firebase backend for reading and writing data.

5. **Google Chrome / Any Modern Web Browser**

Required for accessing the web dashboard.

6. **Operating System**

- For development: Windows 10 or higher / Linux (Ubuntu)
- For deployment: Can be hosted on Firebase Hosting or any compatible cloud server.

7.2 HARDWARE REQUIREMENTS

1. **ESP32 Development Board**

A powerful microcontroller with built-in Wi-Fi and Bluetooth Low Energy (BLE) capabilities. Acts as the BLE receiver at the workplace entrance.

2. **SparkFun NanoBeacon IN100**

Used as the BLE beacon that each employee will carry. Continuously emits a unique UUID to identify the employee.

3. **Coin Cell Batteries (e.g., CR2032)**

Powers the BLE beacons. Chosen for their compact size and long battery life.

4. **USB Cable and Power Adapter**

Required to power the ESP32 board.

5. **Wi-Fi Router / Access Point**

Provides internet connectivity to the ESP32 for transmitting data to Firebase.

6. **Computer or Laptop**

Used for development, programming, and deployment of the system.

To design an effective and practical BLE-based employee tracking system, it was important to understand existing technologies, systems, and research efforts in the field of attendance monitoring and workforce automation. The following sources and studies helped shape the approach and validated the need for such a solution:

1. "Bluetooth Low Energy (BLE) for Industrial Applications" – IEEE Journals

This study explored how BLE can be applied in real-time industrial tracking scenarios. It confirmed BLE's low power consumption and reliable range (10–30 meters), making it suitable for employee detection at entry/exit points.

2. "Time and Attendance Tracking with IoT Devices" – ResearchGate

This paper discussed the inefficiencies of RFID and biometric systems. It proposed IoT-based models that automate attendance tracking and minimize human intervention, thereby enhancing data reliability and reducing fraud.

3. "Firebase as a Scalable Backend for Real-Time Applications" – Firebase Documentation and Developer Blogs

Firebase is widely used in IoT projects due to its real-time data synchronization, easy API access, and integration with web and mobile platforms. It provides robust support for live dashboards and cloud-hosted data.

4. "ESP32 Technical Reference Manual" – Espressif Systems

The ESP32 is a low-cost, feature-rich microcontroller with integrated BLE and Wi-Fi. Its technical documentation was crucial for designing the firmware logic, enabling it to detect beacon signals and connect to Firebase.

5. "IoT-Based Smart Attendance System Using BLE Beacons" – International Journal of Advanced Research in Computer Engineering & Technology (IJARCET)

This paper presented a system similar to our concept. It emphasized the advantages of using BLE beacons over RFID, including passive detection and higher automation. However, their design lacked real-time data sync and web dashboard integration, which our system includes.

These references collectively justify the technical feasibility and growing relevance of BLE-based attendance systems. They reinforce the direction taken in this project, which combines BLE, ESP32, and Firebase to create a robust, scalable, and real-time employee tracking system.

The **BLE-Based Employee Tracking System** aims to automate the attendance and work hour monitoring process for employees using BLE technology. The system provides a scalable, touchless, and low-maintenance alternative to traditional methods such as RFID cards and biometric systems.

Key Functionalities Covered in This Project:

1. BLE Beacon-Based Attendance

Employees are assigned BLE beacons that continuously transmit unique UUID signals. These beacons serve as identity tags for automatic detection.

2. ESP32-Based BLE Receiver

The ESP32 microcontroller detects beacon signals within a defined range (approximately 30 meters) and logs the entry and exit times for each employee.

3. Wi-Fi Connectivity and Firebase Integration

The ESP32 uploads the captured data to a cloud-based backend using Wi-Fi. Firebase is used to store, manage, and process real-time attendance data.

4. Work Hour Calculation

The system calculates the total active time of each employee based on their daily entry and exit logs.

5. Web Dashboard for Monitoring

A user-friendly dashboard is developed using web technologies to allow remote access to employee logs, time summaries, and status updates.

6. Multi-Device Accessibility

The system can be accessed from desktops, laptops, or mobile browsers, enabling flexibility and convenience for site supervisors or HR managers.

Out of Scope:

- Integration with payroll processing systems
- Mobile app development (only web interface is considered)
- BLE beacon battery optimization or power-saving algorithms
- Indoor positioning or real-time location tracking within a site

This system is designed to be **easily expandable**, allowing future upgrades such as notification systems, app-based controls, location tracking, and integration with external attendance or payroll software. It lays a strong foundation for building an end-to-end workforce automation platform for smart industries and digital workplaces.



Figure 10.1. ER Diagram

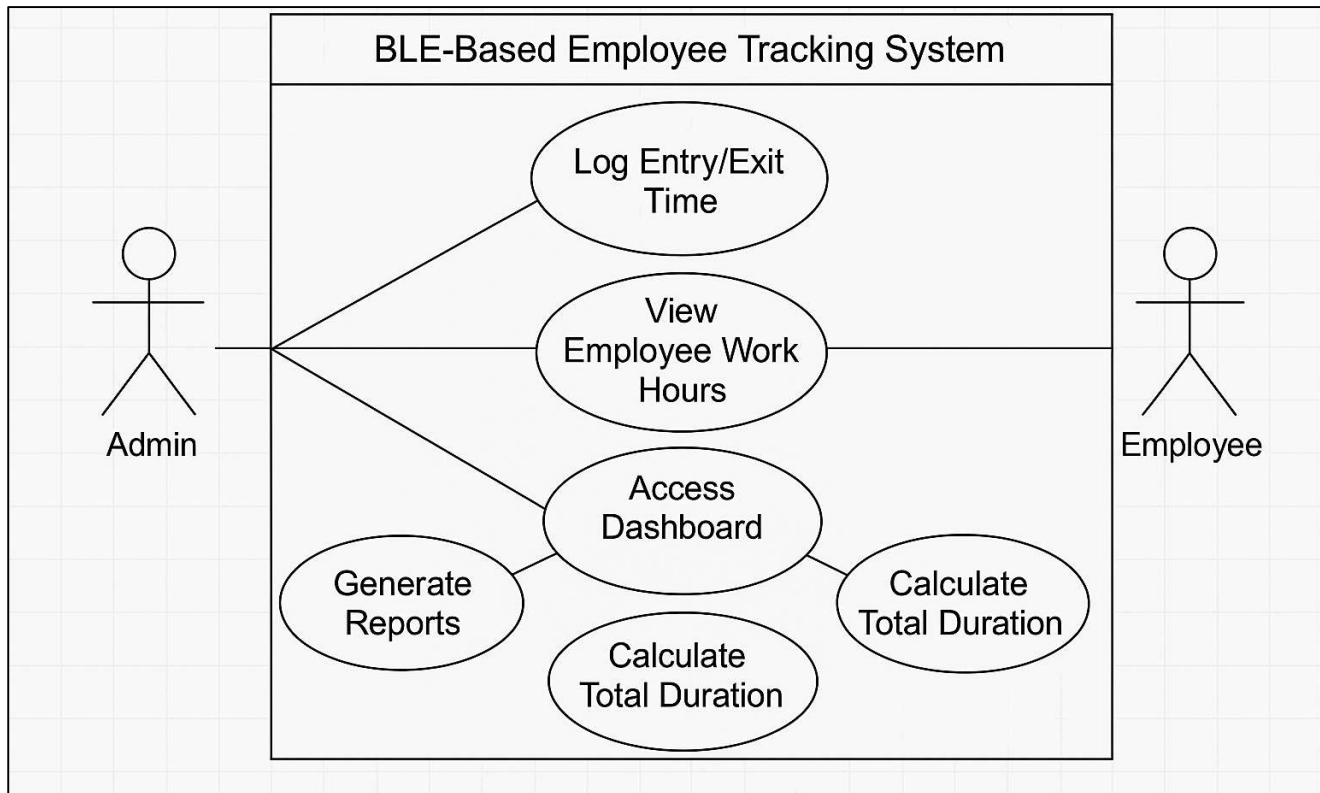


Figure 10.2. USE CASE

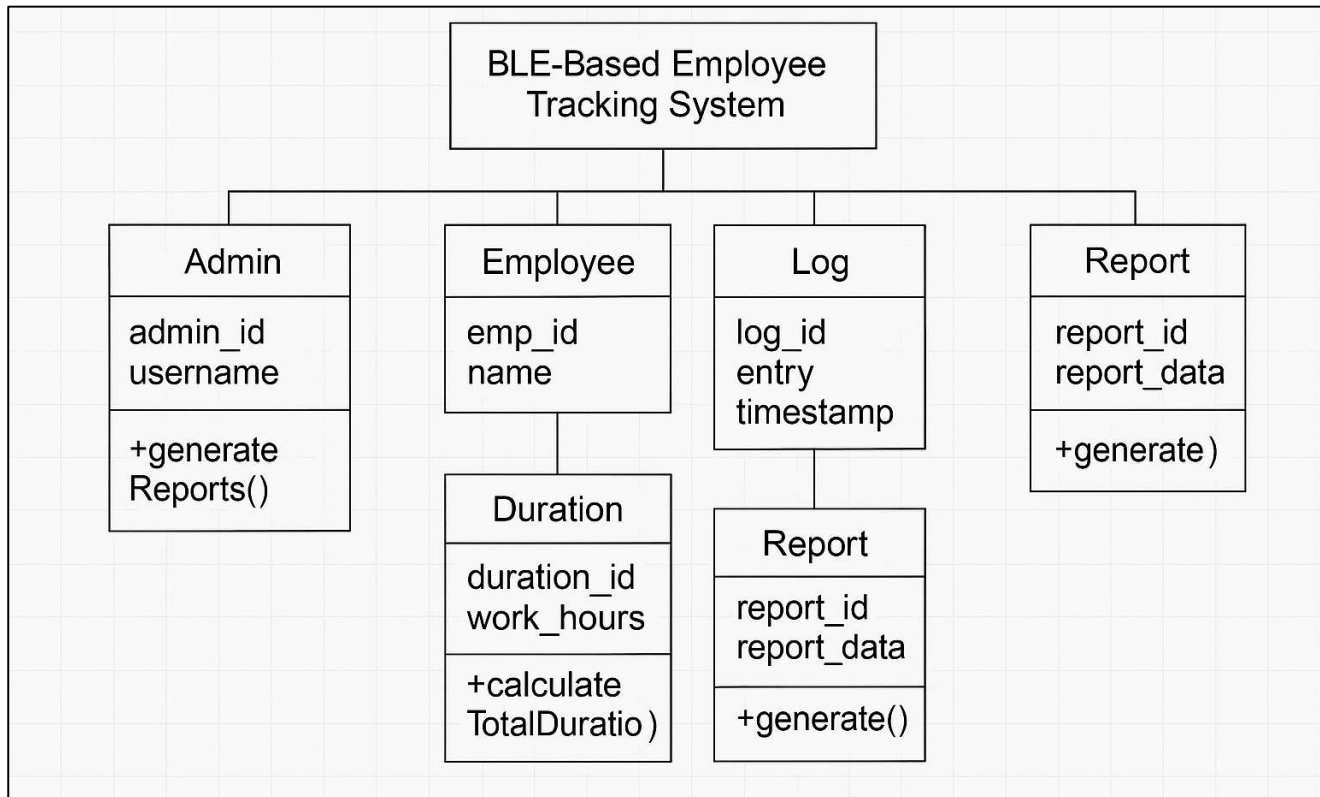


Figure 10.3. Class Diagram

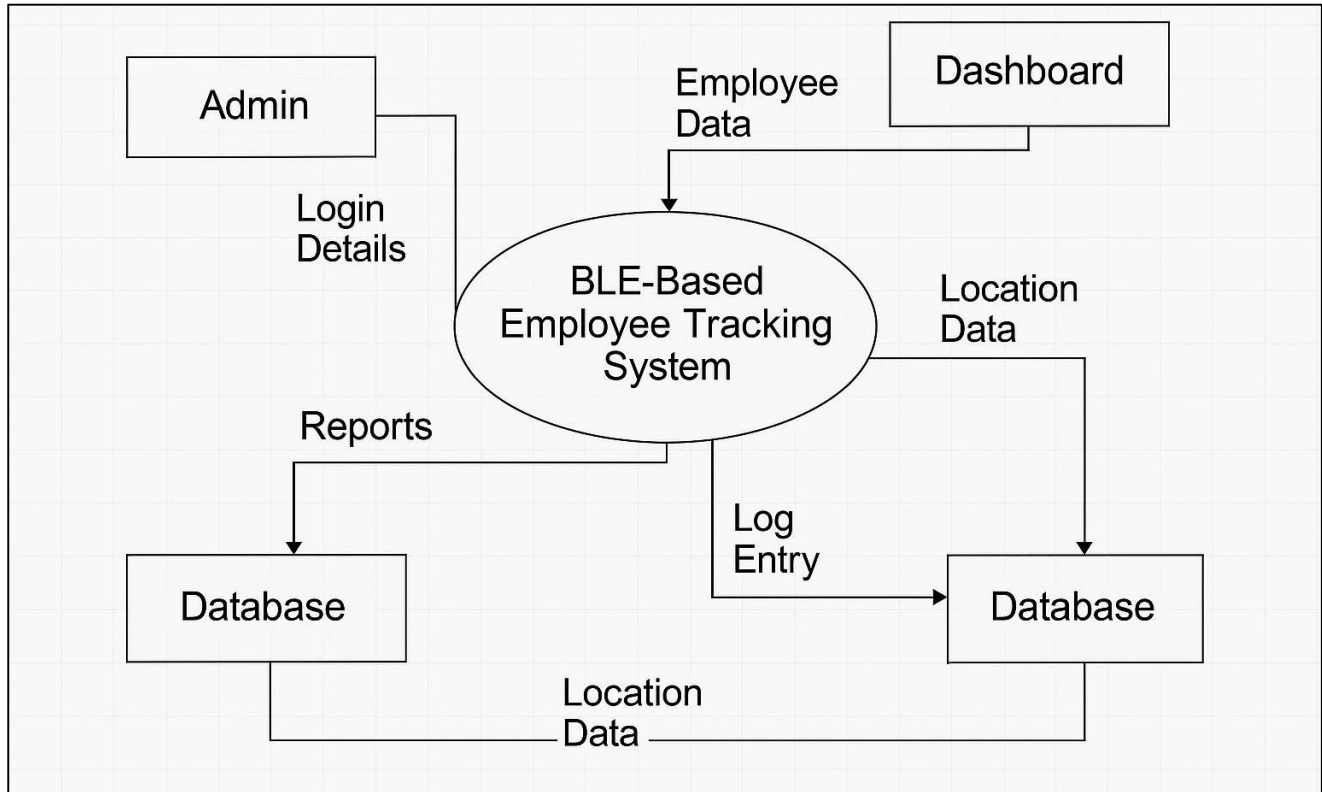


Figure 10.4. Data Flow Diagram

The **BLE-Based Employee Tracking System** offers a range of intelligent features that make it efficient, user-friendly, and highly suitable for modern workforce environments. The system combines IoT technologies like BLE and ESP32 with cloud computing via Firebase, enabling seamless and automated employee tracking.

1. Contactless Attendance Logging

The system enables employees to be automatically logged in and out as they enter or exit the work premises. This reduces the need for physical attendance systems such as biometric scanners or manual registers.

2. Real-Time Detection Using BLE Beacons

Employees carry a BLE beacon that emits a unique signal. These beacons are detected in real-time by the ESP32 receiver placed at the site's entry point, ensuring timely and accurate logging.

3. ESP32 Receiver with Wi-Fi Support

The ESP32 microcontroller, known for its low power consumption and dual connectivity (Wi-Fi and BLE), detects beacon signals and transmits the data to the cloud in real time.

4. Firebase Cloud Integration

All entry and exit logs are stored in Firebase, allowing for real-time updates, seamless synchronization, and secure data storage accessible from anywhere.

5. Automatic Work Hour Calculation

The system intelligently calculates the total active work duration per employee each day using the time difference between entry and exit timestamps.

6. Web Dashboard for Monitoring

A responsive and user-friendly web dashboard provides authorized personnel (admin/supervisors) access to attendance logs, active hours, and status of each employee.

7. Passive Tracking

Unlike RFID or biometric systems, employees don't need to remember to scan a card or press a button. Their presence is passively detected when in range (up to ~30 meters), ensuring seamless operation.

8. Remote Accessibility

Through cloud connectivity, data can be accessed from remote locations, making it suitable for project sites, construction areas, or multiple office locations.

9. Scalable Design

The system can be expanded to accommodate any number of employees and devices by simply adding more BLE beacons and ESP32 receivers without major changes in architecture.

10. Energy Efficiency

BLE beacons are optimized for low energy consumption, allowing them to operate for several months or even a year on a small coin cell battery.

1) ESP32 CODE :-

```
#include <WiFi.h>
#include <FirebaseESP32.h>
#include <BLEDevice.h>
#include <BLEScan.h>
#include <time.h>

// Firebase Configuration
#define FIREBASE_HOST "emprolltracking-default-rtdb.firebaseio.com"
#define FIREBASE_SECRET "tEsqKC0bUzeeyBQ2HCfxFZeynQoUyhX6eEXVu3gT"

// WiFi Credentials
#define WIFI_SSID "BLEPROJECT"
#define WIFI_PASSWORD "Anmol@<"

// BLE Configuration
#define SCAN_TIME 3 // Seconds per scan
#define EXIT_TIMEOUT 30 // 30 seconds without signal = exit
#define DEBOUNCE_TIME 10 // 10 seconds debounce to avoid multiple logs

FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;

struct Employee {
    const char* beaconName;
    const char* empID;
    bool isActive;
    unsigned long lastSeen;
    unsigned long entryTime;
    unsigned long dailyTotal;
};

Employee employees[] = {
    {"Emp1Anmol@comp&iot", "emp1", false, 0, 0, 0},
    {"Emp2Manthan@comp&iot", "emp2", false, 0, 0, 0}
};

BLEScan* pBLEScan;
unsigned long lastReset = 0;

void setup() {
    Serial.begin(115200);

    // Initialize WiFi
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    Serial.print("Connecting to WiFi");
    while (WiFi.status() != WL_CONNECTED) {
        Serial.print(".");
        delay(500);
    }
    Serial.println("\nConnected: " + WiFi.localIP().toString());

    // Initialize Firebase
    config.host = FIREBASE_HOST;
    config.signer.tokens.legacy_token = FIREBASE_SECRET;
    Firebase.begin(&config, &auth);
    Firebase.reconnectWiFi(true);
```

```

// Initialize Time
configTime(19800, 0, "pool.ntp.org"); // IST: GMT+5:30
while(!time(nullptr)) delay(1000);

// Initialize BLE
BLEDevice::init("");
pBLEScan = BLEDevice::getScan();
pBLEScan->setActiveScan(true);
pBLEScan->setInterval(97);
pBLEScan->setWindow(37);
}

void loop() {
    checkDailyReset();
    processBLEScan();
    checkForExits();
    delay(2000);
}

void processBLEScan() {
    BLEScanResults* scanResults = pBLEScan->start(SCAN_TIME, false); // Fixed pointer declaration
    time_t now;
    time(&now);

    if (scanResults) {
        for(int i = 0; i < scanResults->getCount(); i++) { // Use -> operator
            BLEAdvertisedDevice device = scanResults->getDevice(i);
            String devName = device.getName().c_str();

            for(auto &emp : employees) {
                if(devName == emp.beaconName) {
                    if (!emp.isActive) { // Entry event
                        emp.isActive = true;
                        emp.entryTime = now;
                        logFirebaseEvent(emp.empID, "entry", now);
                    }
                    emp.lastSeen = now; // Update last seen time
                }
            }
        }
        pBLEScan->clearResults();
    }
}

void checkForExits() {
    time_t now;
    time(&now);

    for (auto &emp : employees) {
        if (emp.isActive && (now - emp.lastSeen > EXIT_TIMEOUT)) {
            emp.isActive = false;
            emp.dailyTotal += (now - emp.entryTime);
            logFirebaseEvent(emp.empID, "exit", now);
            Firebase.setDouble(fbdo, getFirebasePath(emp.empID) + "/total", emp.dailyTotal);
        }
    }
}

```

```

void logFirebaseEvent(const char* empID, const char* type, time_t timestamp) {
    FirebaseJson json;
    json.add("type", type);
    json.add("timestamp", formatTimestamp(timestamp));

    String path = getFirebasePath(empID) + "/logs";
    Firebase.pushJSON(fbdo, path, json);
}

String getFirebasePath(const char* empID) {
    time_t now;
    time(&now);
    struct tm* timeinfo = localtime(&now);
    char dateStr[11];
    strftime(dateStr, 11, "%Y-%m-%d", timeinfo);
    return "/attendance/" + String(dateStr) + "/" + String(empID);
}

String formatTimestamp(time_t timestamp) {
    struct tm* timeinfo = localtime(&timestamp);
    char buffer[20];
    strftime(buffer, sizeof(buffer), "%Y-%m-%d %H:%M:%S", timeinfo);
    return String(buffer);
}

void checkDailyReset() {
    time_t now;
    time(&now);
    struct tm* timeinfo = localtime(&now);

    if(timeinfo->tm_hour == 0 && timeinfo->tm_min == 0) {
        if(millis() - lastReset > 86400000) { // 24h cooldown
            lastReset = millis();
            resetDailyCounters();
        }
    }
}

void resetDailyCounters() {
    for(auto &emp : employees) {
        emp.dailyTotal = 0;
        Firebase.setDouble(fbdo, getFirebasePath(emp.empID) + "/total", 0);
    }
}

```

Website Codes:

1)login Page:

```
import React, { useState } from "react";
import { auth } from "../firebase";
import { signInWithEmailAndPassword, GoogleAuthProvider, signInWithPopup } from "firebase/auth";
import { useNavigate } from "react-router-dom";
import "../Login.css";
import { IonIcon } from "@ionic/react";
import { mailOutline, lockClosedOutline, eyeOutline, eyeOffOutline } from "ionicons/icons";

function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [showPassword, setShowPassword] = useState(false); // State for password visibility
  const navigate = useNavigate();

  const handleLogin = async (e) => {
    e.preventDefault();
    try {
      await signInWithEmailAndPassword(auth, email, password);
      navigate("/home");
    } catch (err) {
      setError(err.message || "An error occurred while logging in.");
    }
  };

  const handleGoogleSignIn = async () => {
    const provider = new GoogleAuthProvider();
    try {
      await signInWithPopup(auth, provider);
      navigate("/home");
    } catch (err) {
      setError(err.message || "An error occurred while signing in with Google.");
    }
  };

  // Toggle password visibility
  const togglePasswordVisibility = () => {
    setShowPassword(!showPassword);
  };

  return (
    <div className="screen-1">
      { /* EMPTRACK Text */ }
      <h1 className="logo-text">EMPTRACK</h1>

      { /* Email Input */ }
      <div className="email">
        <label htmlFor="email">Email Address</label>
        <div className="sec-2">
          <IonIcon icon={mailOutline} />
          <input
            type="email"
            name="email"
            placeholder="Username@gmail.com"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            required
          />
        </div>
      </div>
    </div>
  );
}
```

```

    { /* Password Input */ }
    <div className="password">
      <label htmlFor="password">Password</label>
      <div className="sec-2">
        <IonIcon icon={lockClosedOutline} />
        <input
          type={showPassword ? "text" : "password"} // Toggle between text and password
          name="password"
          placeholder="....."
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          required
        />
        <IonIcon
          icon={showPassword ? eyeOffOutline : eyeOutline} // Change icon based on visibility
          className="show-hide"
          onClick={togglePasswordVisibility} // Toggle password visibility
        />
      </div>
    </div>

    { /* Login Button */ }
    <button className="login" onClick={handleLogin}>
      Login
    </button>

    { /* Google Sign-In Button */ }
    <button className="google-btn" onClick={handleGoogleSignIn}>
      Sign in with Google
    </button>

    { /* Error Message */ }
    {error && <p className="error">{error}</p>}

    { /* Footer Links */ }
    <div className="footer">
      <span>Sign up</span>
      <span>Forgot Password?</span>
    </div>
  </div>
);
}

export default Login;

```

2)Dashboard Page

```
import React, { useEffect, useState } from "react";
import { database } from "../firebase";
import { ref, onValue } from "firebase/database";
import { Link, useNavigate } from "react-router-dom";
import NavBar from "../Navbar"; // Import NavBar component

function Home() {
  // State variables
  const [activeEmployees, setActiveEmployees] = useState([]);
  const [inactiveEmployees, setInactiveEmployees] = useState([]);
  const [menu, setMenu] = useState("home");
  const [activeTab, setActiveTab] = useState("present");
  const navigate = useNavigate();

  useEffect(() => {
    // Retrieve today's date using local time
    const today = () => {
      const date = new Date();
      const year = date.getFullYear();
      const month = String(date.getMonth() + 1).padStart(2, '0');
      const day = String(date.getDate()).padStart(2, '0');
      return `${year}-${month}-${day}`;
    };

    const attendanceRef = ref(database, `attendance/${today}`);

    // Listener for attendance data
    onValue(attendanceRef, (snapshot) => {
      const data = snapshot.val();
      if (!data) return;

      const active = [];
      const inactive = [];
      for (const empID in data) {
        const logs = data[empID].logs;
        const lastLog = Object.values(logs).pop();

        // Categorize employees as active or inactive
        lastLog.type === "entry" ? active.push(empID) : inactive.push(empID);
      }
      setActiveEmployees(active);
      setInactiveEmployees(inactive);
    });
  }, []);

  // Log out function
  const logOut = () => {
    console.log("Logged out");
    navigate("/login");
  };

  return (
    <div style={styles.container}>
      { /* NavBar */ }
      <div style={styles.navbar}>
        <NavBar menu={menu} setmenu={setMenu} logOut={logOut} />
      </div>

      { /* Main Content */ }
      <div style={styles.content}>
        <h1 style={styles.dashboardTitle}>Dashboard</h1>
        { /* Statistics Cards */ }
        <div style={styles.statsContainer}>
          <StatsCard title="Total Employees" value={activeEmployees.length + inactiveEmployees.length} color="#007bff" />
        </div>
      </div>
    </div>
  );
}
```

```

        <StatsCard title="Present Employees" value={activeEmployees.length} color="#28a745" />
        <StatsCard title="Absent Employees" value={inactiveEmployees.length} color="#dc3545" />
    </div>

    {/ * Tabs and Employee List */}
    <div style={styles.tabContainer}>
        {/ * Tab Buttons */}
        <div style={styles.tabButtons}>
            <TabButton active={activeTab === "present"} color="#007bff" onClick={() => setActiveTab("present")}
text="Present Employees" />
            <TabButton active={activeTab === "absent"} color="#dc3545" onClick={() => setActiveTab("absent")}
text="Absent Employees" />
        </div>

        {/ * Employee List */}
        <div style={styles.employeeList}>
            {(activeTab === "present" ? activeEmployees : inactiveEmployees).map((emp, index) => (
                <EmployeeCard key={index} empID={emp} />
            ))}
        </div>
    </div>
</div>
);
}

import { Link } from "react-router-dom";

// Stats Card Component
const StatsCard = ({ title, value, color }) => (
    <div style={{ ...styles.statsCard, borderLeft: `5px solid ${color}` }}>
        <h3 style={styles.cardTitle}>{title}</h3>
        <p style={{ ...styles.cardValue, color }}>{value}</p>
    </div>
);

// Employee Card Component
const EmployeeCard = ({ empID }) => (
    <Link to={`/employee/${empID}`} style={styles.linkStyle}>
        <div style={styles.employeeCard}>
            <h3 style={styles.employeeName}>{empID}</h3>
        </div>
    </Link>
);

// Tab Button Component
const TabButton = ({ active, color, onClick, text }) => (
    <button
        style={{
            ...styles.tabButton,
            backgroundColor: active ? color : "#f4f4f9",
            color: active ? "fff" : "#000"
        }}
        onClick={onClick}
    >
        {text}
    </button>
);

```

```

// Styles
const styles = {
  container: {
    display: "flex",
    height: "100vh",
    fontFamily: "Arial, sans-serif",
    margin: 0
  },
  navbar: {
    width: "70px",
    backgroundColor: "#f4f4f9",
    padding: "20px",
    display: "flex",
    flexDirection: "column",
    alignItems: "center",
    boxShadow: "2px 0 4px rgba(0, 0, 0, 0.1)",
    position: "fixed",
    top: 0,
    bottom: 0,
    left: 0,
    overflowY: "auto"
  },
  content: {
    marginLeft: "90px",
    flex: 1,
    padding: "20px",
    overflowY: "auto",
    boxSizing: "border-box"
  },
  dashboardTitle: {
    fontSize: "2rem",
    fontWeight: "bold",
    marginBottom: "20px",
    position: "absolute",
    top: "20px",
    left: "120px"
  },
  statsContainer: {
    display: "flex",
    gap: "10px",
    marginBottom: "20px",
    marginTop: "50px"
  },
  statsCard: {
    backgroundColor: "#fff",
    padding: "20px",
    borderRadius: "8px",
    boxShadow: "0 2px 4px rgba(0, 0, 0, 0.1)",
    width: "418px",
    textAlign: "center"
  },
  cardTitle: {
    color: "#000",
    fontSize: "1rem",
    fontWeight: "500"
  },
  cardValue: {
    fontSize: "2rem",
    fontWeight: "bold"
  },
  tabContainer: {
    backgroundColor: "#fff",
    padding: "20px",
    borderRadius: "20px",
    boxShadow: "0 2px 4px rgba(0, 0, 0, 0.1)"
  }
}

```

```

    },
    tabButtons: {
      display: "flex",
      justifyContent: "space-between",
      marginBottom: "20px"
    },
    tabButton: {
      padding: "10px 20px",
      border: "none",
      borderRadius: "50px",
      cursor: "pointer",
      fontSize: "1rem",
      fontWeight: "500",
      flex: 1,
      marginRight: "10px",
      transition: "background-color 0.3s ease"
    },
    employeeList: {
      display: "flex",
      flexWrap: "wrap",
      gap: "20px",
      justifyContent: "center"
    },
    employeeCard: {
      backgroundColor: "#f4f4f4",
      padding: "25px",
      borderRadius: "15px",
      boxShadow: "0 4px 8px rgba(0, 0, 0, 0.1)",
      transition: "box-shadow 0.4s ease",
      width: "280px",
      display: "flex",
      justifyContent: "center",
      alignItems: "center",
      textAlign: "center",
      cursor: "pointer"
    },
    employeeName: {
      fontSize: "1.2rem",
      fontWeight: "600",
      margin: 0
    },
    linkStyle: {
      textDecoration: "none",
      color: "inherit"
    }
  }
};

const Home = () => (
  <div style={styles.container}>
    /* Add Navbar, Dashboard, etc. */
  </div>
);

export default Home;

```

3)EmployeeDetails Page:

```
import React, { useEffect, useState } from "react";
import { database } from "../firebase";
import { ref, onValue } from "firebase/database";
import { useParams } from "react-router-dom";
import DatePicker from "react-datepicker";
import "react-datepicker/dist/react-datepicker.css";
import "../EmployeeDetails.css";

function EmployeeDetails() {
  const { id: empID } = useParams();
  const [details, setDetails] = useState({
    activeHoursToday: 0,
    presentDays: 0,
    avgActiveHours: 0,
  });
  const [selectedDate, setSelectedDate] = useState(null);
  const [dateDetails, setDateDetails] = useState({
    activeHours: 0,
    logs: [],
  });
  const [showMonthDropdown, setShowMonthDropdown] = useState(false);
  const [selectedMonthForExport, setSelectedMonthForExport] = useState("");

  const formatTimestampTo12Hour = (timestamp) => {
    const date = new Date(timestamp);
    let hours = date.getHours();
    const minutes = date.getMinutes();
    const ampm = hours >= 12 ? "PM" : "AM";
    hours = hours % 12 || 12;
    const formattedMinutes = minutes < 10 ? `0${minutes}` : minutes;
    return `${hours}.${formattedMinutes} ${ampm}`;
  };

  const calculateDuration = (entryTime, exitTime) => {
    const parseTime = (timeStr) => {
      const [time, period] = timeStr.split(' ');
      let [hours, minutes] = time.split(':').map(Number);
      if (period === 'PM' && hours !== 12) hours += 12;
      if (period === 'AM' && hours === 12) hours = 0;
      return hours * 60 + minutes;
    };

    const entryMinutes = parseTime(entryTime);
    const exitMinutes = parseTime(exitTime);
    const totalMinutes = exitMinutes - entryMinutes;
    const hours = Math.floor(totalMinutes / 60);
    const minutes = totalMinutes % 60;
    return `${hours}h ${minutes}m`;
  };

  useEffect(() => {
    const now = new Date();
    const year = now.getFullYear();
    const month = String(now.getMonth() + 1).padStart(2, "0");
    const today = `${year}-${month}-${String(now.getDate()).padStart(2, "0")}`;
    const firstDayOfMonth = `${year}-${month}-01`;
    const attendanceRef = ref(database, "attendance");

    onValue(attendanceRef, (snapshot) => {
      const data = snapshot.val();
      if (!data) return;

      let totalActiveHoursToday = 0;
```

```

let totalPresentDays = 0;
let totalActiveHoursMonth = 0;

for (const date in data) {
  if (date >= firstDayOfMonth && date <= today && data[date][empID]) {
    const dailyTotal = data[date][empID].total || 0;
    if (dailyTotal > 0) totalPresentDays++;
    totalActiveHoursMonth += dailyTotal;
    if (date === today) {
      totalActiveHoursToday = dailyTotal;
    }
  }
}

const avgActiveHours =
  totalPresentDays > 0
    ? (totalActiveHoursMonth / totalPresentDays / 3600).toFixed(2)
    : 0;

setDetails({
  activeHoursToday: (totalActiveHoursToday / 3600).toFixed(2),
  presentDays: totalPresentDays,
  avgActiveHours: avgActiveHours,
});
}, [empID]);

useEffect(() => {
  if (!selectedDate) return;
  const formattedDate = selectedDate.toLocaleDateString("en-CA");
  const attendanceRef = ref(database, `attendance/${formattedDate}/${empID}`);

  onValue(attendanceRef, (snapshot) => {
    const data = snapshot.val();
    if (!data) {
      setDateDetails({ activeHours: 0, logs: [] });
      return;
    }

    const logs = Object.values(data.logs || {}).map((log) => ({
      ...log,
      timestamp: formatTimestampTo12Hour(log.timestamp),
    }));

    const activeHours = ((data.total || 0) / 3600).toFixed(2);
    setDateDetails({
      activeHours: activeHours,
      logs: logs,
    });
  });
}, [selectedDate, empID]);

const handleExportMonthlyData = () => {
  if (!selectedMonthForExport) {
    alert("Please select a valid month.");
    return;
  }

  const [year, month] = selectedMonthForExport.split("-");
  const firstDayOfMonth = `${year}-${month}-01`;
  const lastDayOfMonth = `${year}-${month}-${new Date(year, month, 0).getDate()}`;
  const attendanceRef = ref(database, "attendance");

  onValue(attendanceRef, (snapshot) => {
    const data = snapshot.val();

```

```

    if (!data) {
        alert("No data available for the selected month.");
        return;
    }

    const csvRows = ["Date,Active Hours (Hours),Entry/Exit Logs"];
    for (const date in data) {
        if (date >= firstDayOfMonth && date <= lastDayOfMonth && data[date][empID]) {
            const dailyTotal = data[date][empID].total || 0;
            const logs = Object.values(data[date][empID].logs || {});
            const activeHours = (dailyTotal / 3600).toFixed(2);
            const formattedLogs = logs
                .map((log) => `${formatTimestampTo12Hour(log.timestamp)} - ${log.type}`)
                .join(", ");
            csvRows.push(`${date},${activeHours},${formattedLogs}`);
        }
    }

    const csvContent = csvRows.join("\n");
    const blob = new Blob([csvContent], { type: "text/csv" });
    const url = URL.createObjectURL(blob);
    const a = document.createElement("a");
    a.href = url;
    a.download = `${empID}-${year}-${month}-monthly-logs.csv`;
    a.click();
});

setSelectedMonthForExport("");
setShowMonthDropdown(false);
};

const generateMonths = () => {
    const months = [];
    const now = new Date();
    const currentYear = now.getFullYear();
    const currentMonth = now.getMonth() + 1;

    for (let year = currentYear - 1; year <= currentYear; year++) {
        for (let month = 1; month <= 12; month++) {
            if (year === currentYear && month > currentMonth) break;
            months.push(`${year}-${String(month).padStart(2, "0")}`);
        }
    }
    return months;
};

return (
    <div className="container">
        <div className="content">
            <div className="employeeHeader">
                <h2>Employee Details: {empID}</h2>
                <DatePicker
                    selected={selectedDate}
                    onChange={({date}) => setSelectedDate(date)}
                    dateFormat="yyyy-MM-dd"
                    placeholderText="Select a date"
                    className="calendar-button"
                />
            </div>

            <div className="statsContainer">
                <StatsCard title="Total Active Hours Today" value={details.activeHoursToday || 0} />
                <StatsCard title="Total Present Days (This Month)" value={details.presentDays || 0} />
                <StatsCard title="Average Active Hours (This Month)" value={details.avgActiveHours || 0} />
            </div>
        </div>
    </div>
);

```

```

{selectedDate && (
  <div className="tabContainer">
    <h3>Details for {selectedDate.toLocaleDateString("en-CA")}</h3>
    <p>
      <strong>Active Hours:</strong> {dateDetails.activeHours || 0} hours
    </p>
    <h4>Entry/Exit Logs:</h4>
    <div className="logsContainer">
      {dateDetails.logs.reduce((pairs, log, index) => {
        if (log.type === "entry") {
          const exitLog = dateDetails.logs[index + 1]?.type === "exit"
            ? dateDetails.logs[index + 1]
            : null;
          pairs.push({ entry: log, exit: exitLog });
        }
        return pairs;
      }, []).map((pair, index) => (
        <div key={index} className="logCard">
          <div className="logCardHeader">
            <span className="sessionBadge">Session {index + 1}</span>
            <span className="durationBadge">
              {pair.exit ?
                calculateDuration(pair.entry.timestamp, pair.exit.timestamp) :
                'Ongoing'}
            </span>
          </div>
          <div className="logCardBody">
            <div className="logEntry">
              <span className="logType entry">Entry</span>
              <span className="logTime">{pair.entry.timestamp}</span>
            </div>
            {pair.exit && (
              <div className="logEntry">
                <span className="logType exit">Exit</span>
                <span className="logTime">{pair.exit.timestamp}</span>
              </div>
            )}
          </div>
        </div>
      ))}
    </div>
  </div>
)}

<div className="exportSection">
  <button onClick={() => setShowMonthDropdown(!showMonthDropdown)}>
    Export Monthly Data
  </button>
  {showMonthDropdown && (
    <div className="exportControls">
      <select
        className="monthDropdown"
        value={selectedMonthForExport}
        onChange={(e) => setSelectedMonthForExport(e.target.value)}
      >
        <option value="">-- Select Month --</option>
        {generateMonths().map((month) => (
          <option key={month} value={month}>
            {month}
          </option>
        ))}
      </select>
      <button onClick={handleExportMonthlyData}>Export</button>
    </div>
  )}
}

```

```

        </div>
      </div>
    </div>
  );
}

const StatsCard = ({ title, value }) => (
  <div className="statsCard">
    <h3 className="cardTitle">{title}</h3>
    <p className="cardValue">{value}</p>
  </div>
);

export default EmployeeDetails;

```

4)Firebase Configuration:-

```
// Import Firebase modules
import { initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";
import { getDatabase } from "firebase/database";

// Your Firebase configuration
const firebaseConfig = {
  apiKey: "AIzaSyAmA7B_mNFf-gFc38iqJ8NGYB-agCFoeco",
  authDomain: "emprolltracking.firebaseio.com",
  databaseURL: "https://emprolltracking-default-rtdb.firebaseio.com",
  projectId: "emprolltracking",
  storageBucket: "emprolltracking.firebaseio.com",
  messagingSenderId: "822716527493",
  appId: "1:822716527493:web:645c363f54adf8c767caec"
};

// Initialize Firebase
const app = initializeApp(firebaseConfig);

// Export Firebase services
export const auth = getAuth(app);
export const database = getDatabase(app);
```

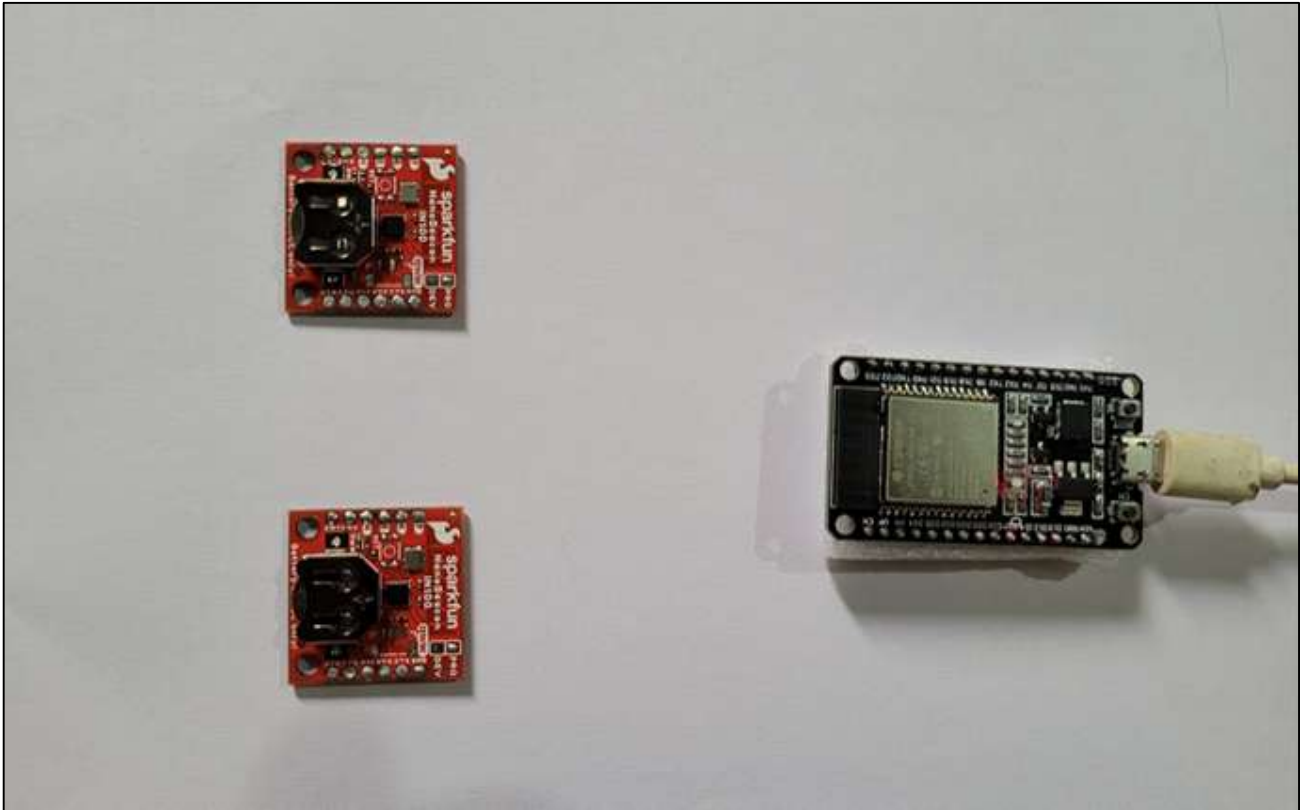


Figure 13.1. Hardware Preview

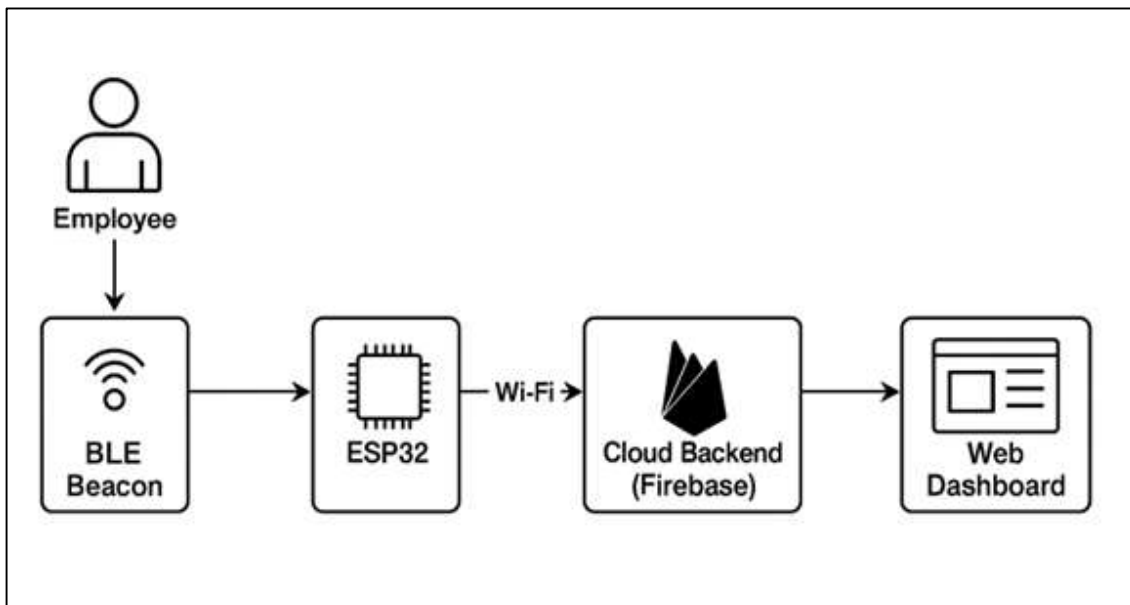


Figure 13.2. Working of Hardware

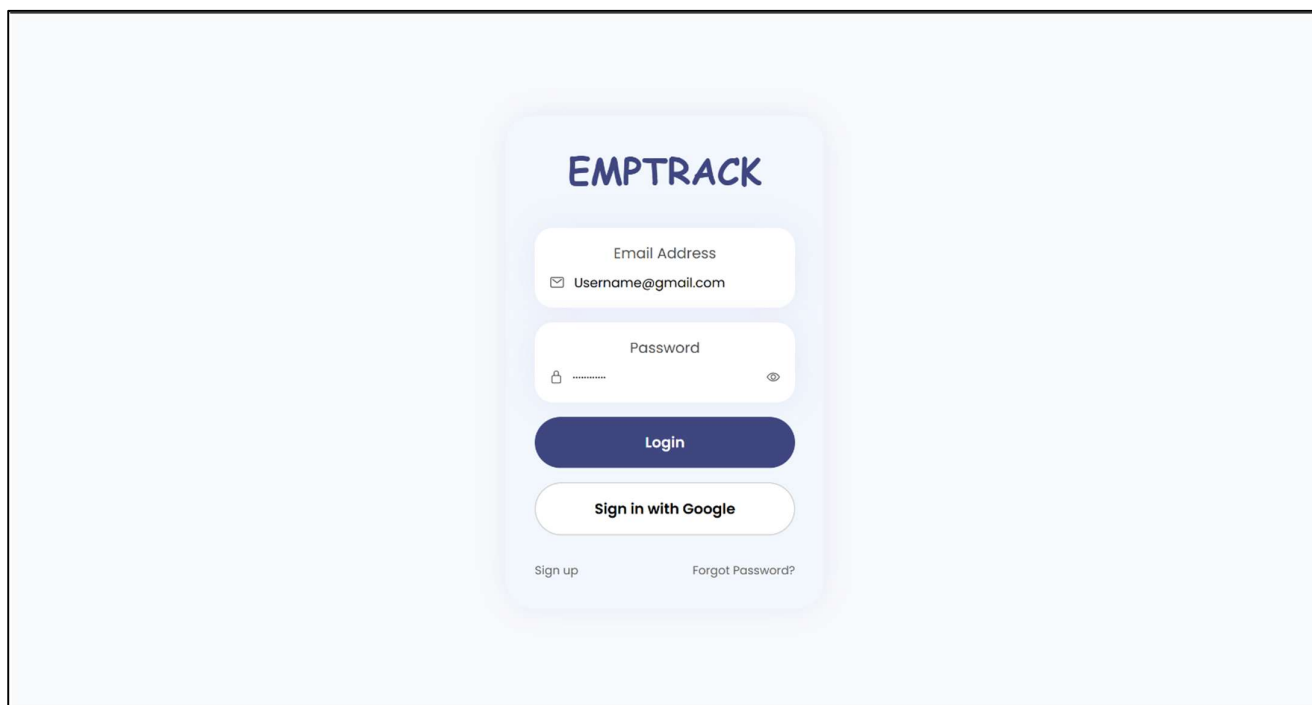


Figure 14.1. Login Page

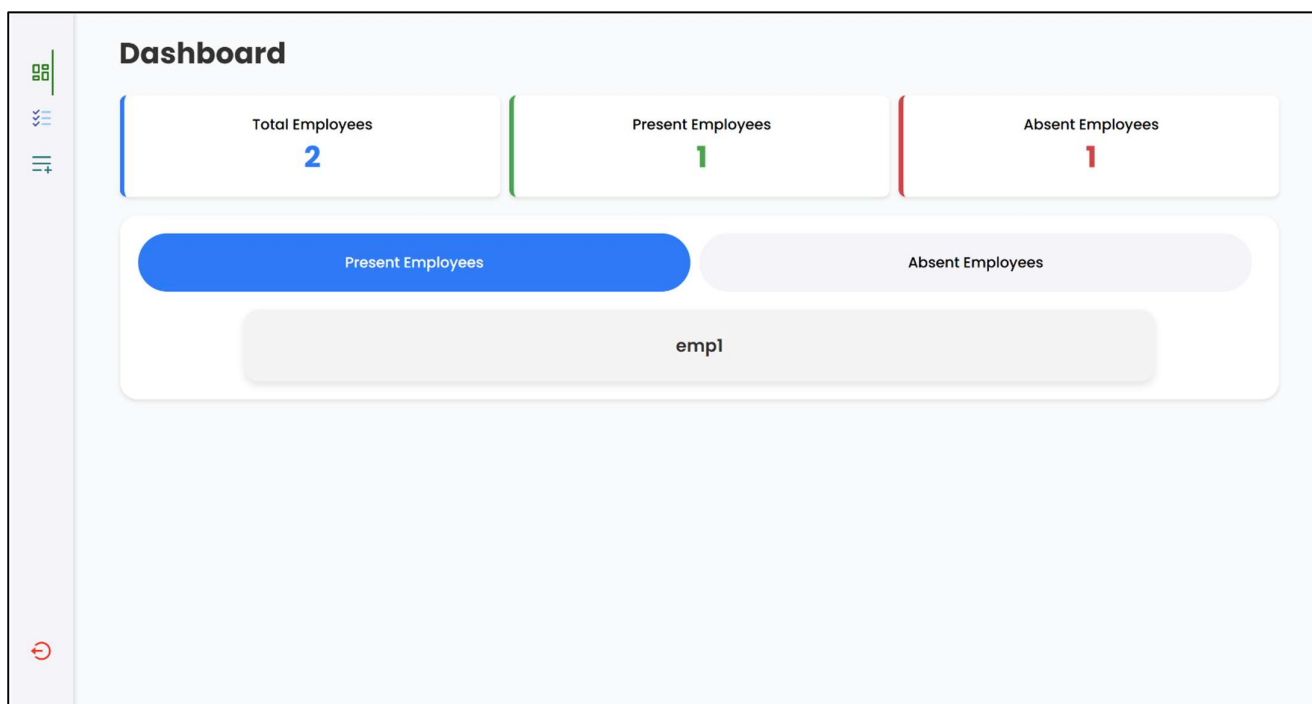


Figure 14.2. Dashboard

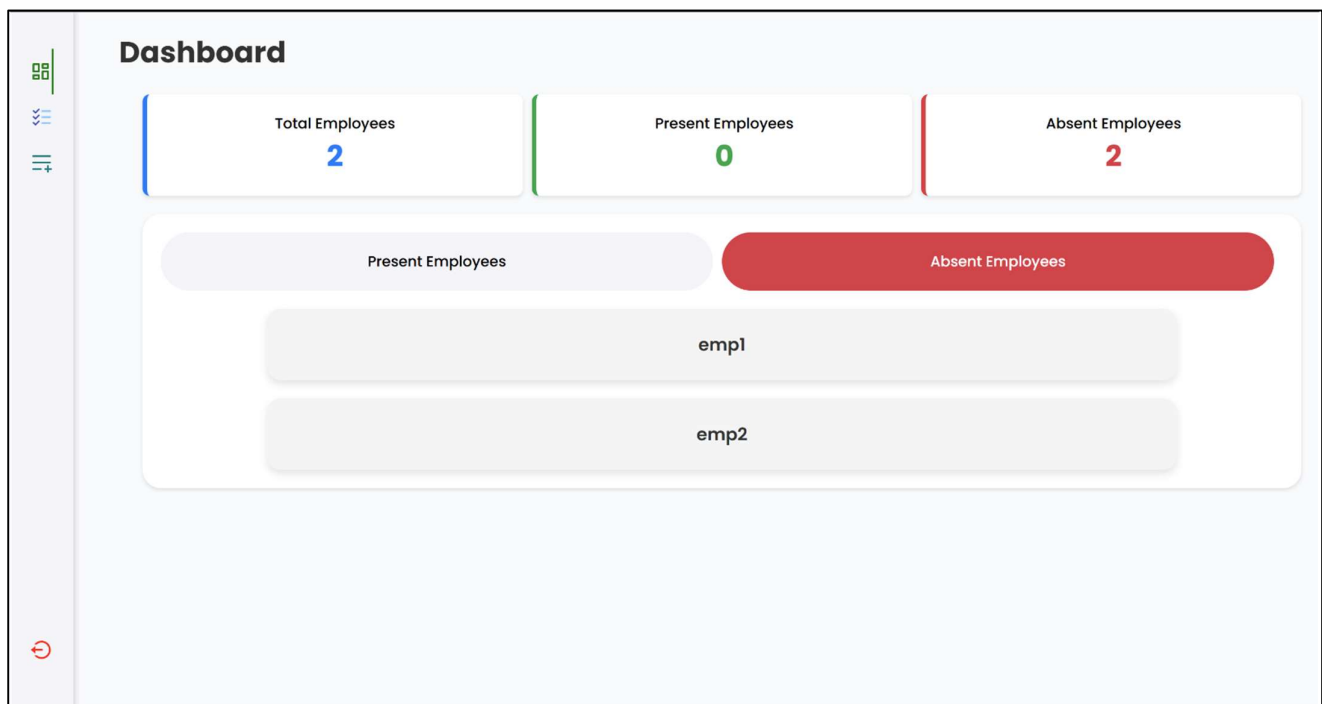
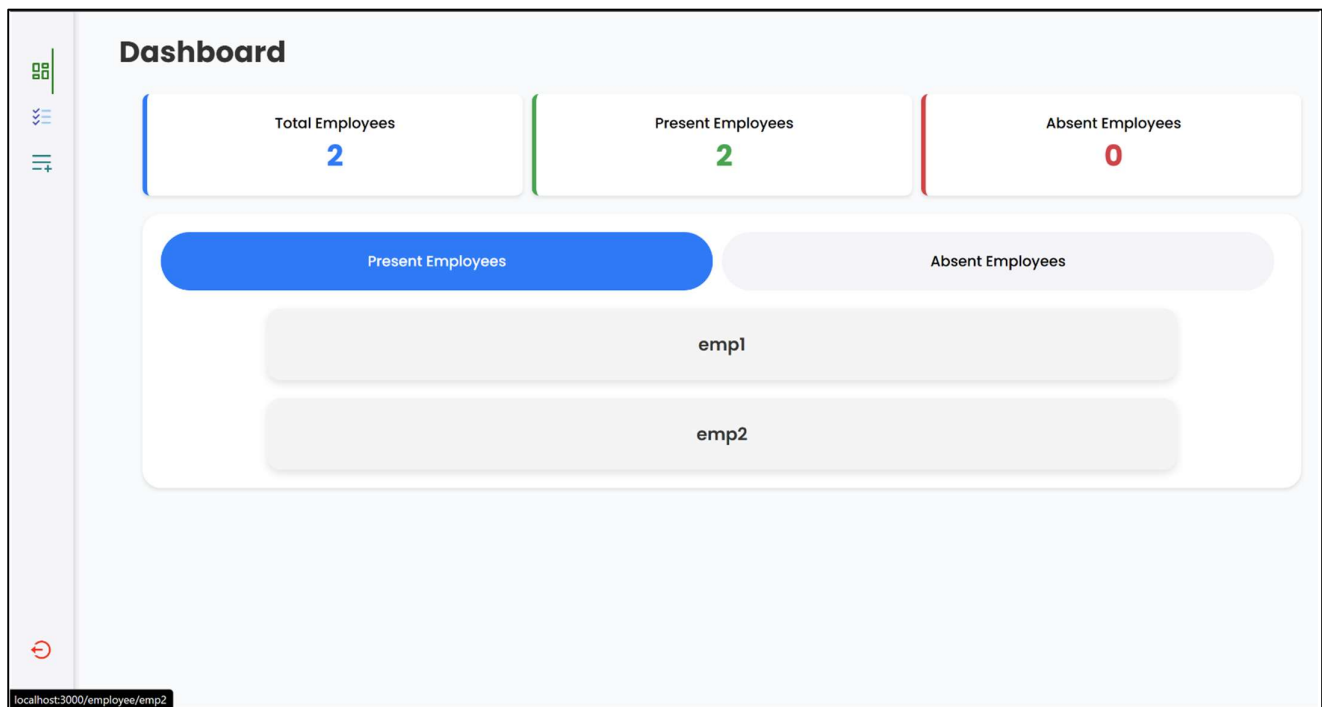


Figure 14.3. Dashboard

Employee Details: emp1

Select a date

Total Active Hours Today

0.36

Total Present Days (This Month)

2

Average Active Hours (This Month)

0.25

Export Monthly Data

Employee Details: emp2

Select a date

Total Active Hours Today

0.06

Total Present Days (This Month)

2

Average Active Hours (This Month)

Export Monthly Data

May 2025						
Su	Mo	Tu	We	Th	Fr	Sa
27	28	29	30	1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

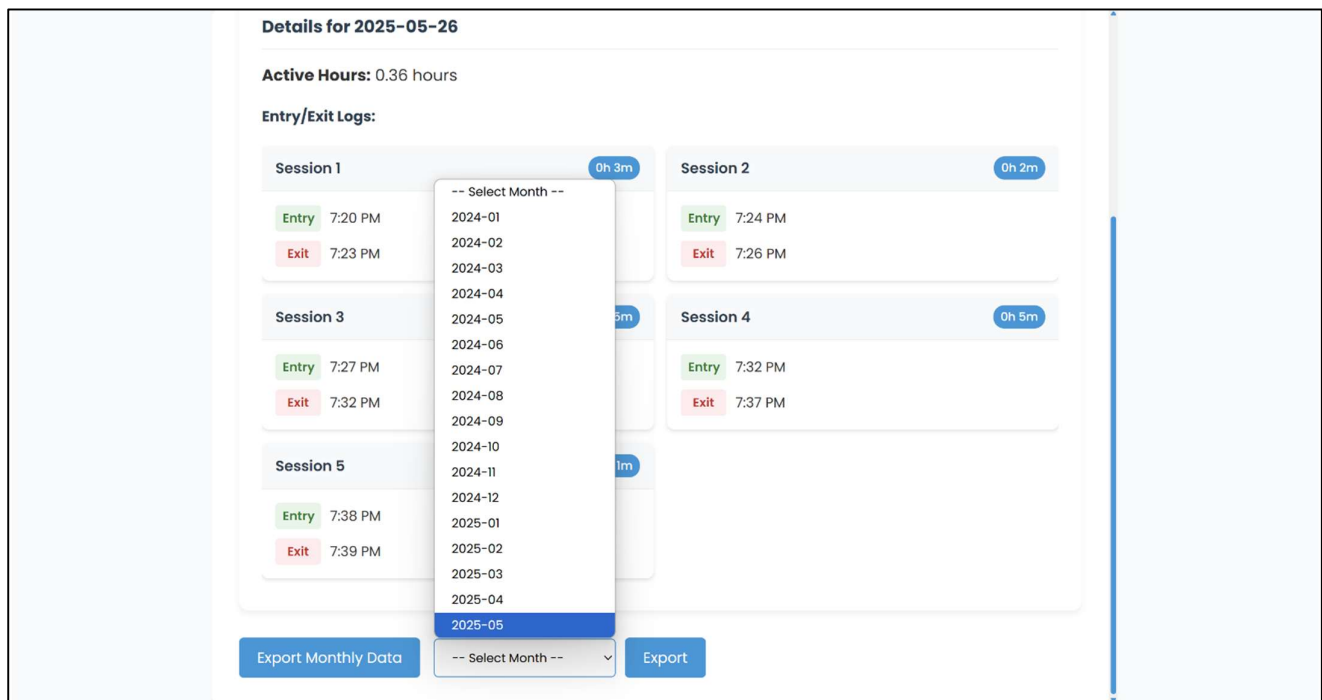
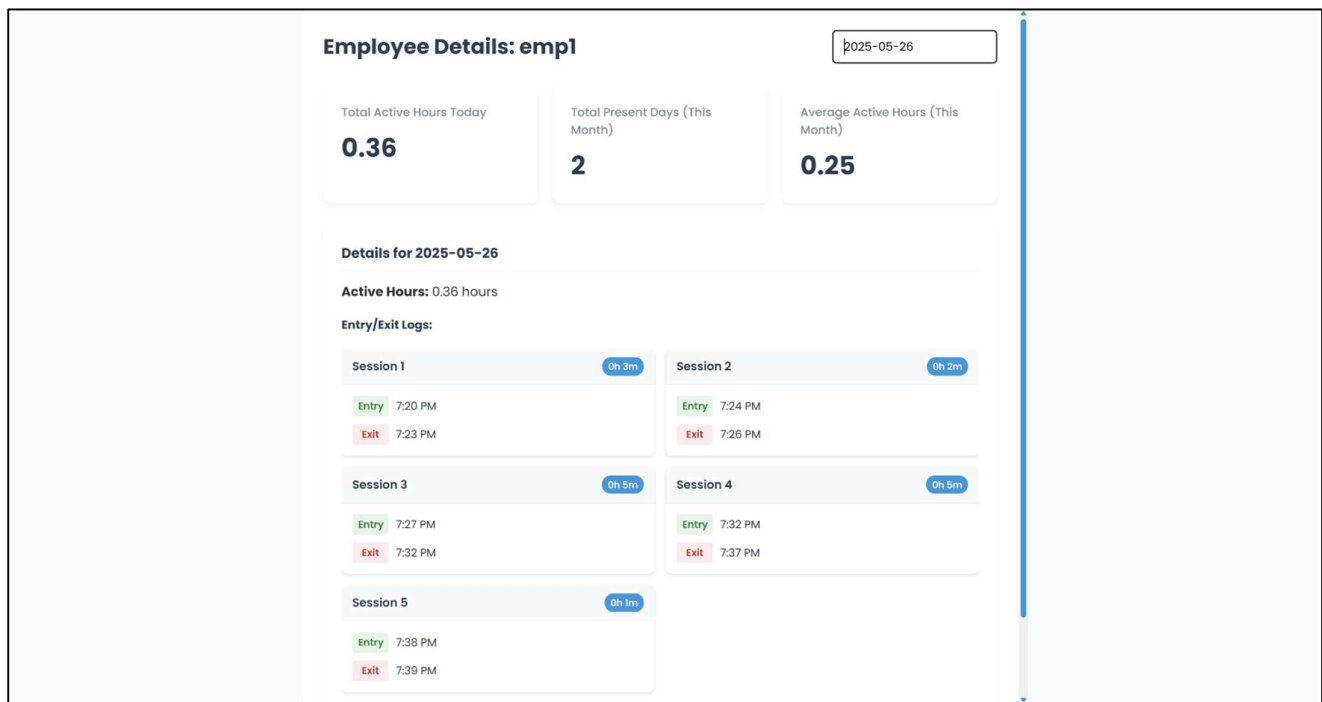


Figure 14.4. EmployeeDetails Page

The **BLE-Based Employee Tracking System** has strong potential for future development, expansion, and real-world implementation across various sectors. While the current version addresses key challenges in employee attendance and work hour monitoring, it also lays the foundation for multiple future enhancements that could further increase its value and impact.

1. Integration with Payroll Systems

The system can be extended to integrate directly with payroll software to automate salary processing based on actual work hours logged. This would eliminate manual calculations and reduce payroll errors.

2. Mobile App for Employee Self-Service

Developing a mobile application for employees would allow them to check their attendance records, receive notifications, and request regularization for missed punches, offering more transparency and engagement.

3. Location-Based Geofencing

Future versions can include geofencing features to restrict logging to specific locations. This would ensure that only authorized areas are used for attendance tracking, improving accuracy and security.

4. Battery Optimization for BLE Beacons

Research and improvements in firmware can help further optimize power consumption, extending beacon battery life beyond a year without compromising signal strength or reliability.

5. AI-Based Productivity Analytics

Integration of AI algorithms could analyze employee work patterns, entry/exit trends, and generate insightful productivity reports, helping managers in performance evaluation and resource planning.

6. Cross-Site Monitoring

The system can be scaled for organizations with multiple sites or branches. A centralized dashboard can be built to monitor attendance across all locations in real time from a single interface.

7. Offline Data Buffering in ESP32

In case of temporary Wi-Fi outages, ESP32 modules could be programmed to locally store attendance logs and sync them to Firebase once the connection is restored.

8. Security Enhancements

Security measures such as data encryption, beacon spoofing prevention, and secure login/authentication methods can be implemented for safe and reliable operation in enterprise environments.

The **BLE-Based Employee Tracking System** is a modern and practical solution aimed at automating the attendance and work-hour tracking process in organizations. By leveraging **Bluetooth Low Energy (BLE)** beacons and **ESP32 microcontrollers**, the system provides a contactless, efficient, and real-time method to monitor employee entry and exit times.

The integration of **Firestore** as the backend and the implementation of a **web dashboard** ensures that data is accessible remotely and in real-time, providing enhanced transparency and ease of management for supervisors and administrators. The system reduces manual errors, eliminates the dependency on traditional attendance methods like biometric scanners or RFID cards, and saves administrative effort and time.

During the development of this project, we not only applied technical skills related to embedded systems, web development, and cloud integration but also gained valuable experience in real-world problem solving, system design, and collaborative teamwork.

This system is scalable, energy-efficient, and adaptable to different environments, from small offices to large industrial or construction sites. It has a solid foundation that can be extended in the future with features like payroll integration, mobile apps, and AI-based analytics.

In conclusion, this project demonstrates how a well-designed IoT-based system can significantly improve operational efficiency in workforce management. It aligns with the vision of smart industry solutions and serves as a stepping stone toward more intelligent and connected workplaces.

1. Espressif Systems. (2023). *ESP32 Technical Reference Manual*. Retrieved from <https://www.espressif.com>
2. Firebase by Google. (2023). *Firebase Documentation*. Retrieved from <https://firebase.google.com/docs>
3. SparkFun Electronics. (2023). *NanoBeacon IN100 Product Page*. Retrieved from <https://www.sparkfun.com>
4. Bluetooth SIG. (2023). *Bluetooth Low Energy (BLE) Overview*. Retrieved from <https://www.bluetooth.com/learn-about-bluetooth/>
5. IEEE Xplore Digital Library. (2022). *Application of BLE for Real-Time Industrial Tracking Systems*. DOI: 10.1109/ACCESS.2022.3154100
6. ResearchGate. (2021). *IoT-Based Smart Attendance Systems: A Comparative Study of RFID, Biometric, and BLE*. Retrieved from <https://www.researchgate.net>
7. GitHub Repository – Firebase Admin SDK. (2023). *Firebase Admin SDK for JavaScript*. Retrieved from <https://github.com/firebase/firebase-admin-node>
8. W3Schools. (2023). *HTML, CSS, and JavaScript Tutorials*. Retrieved from <https://www.w3schools.com>